

STL: Iterator, Containers, Algorithms

林致宇

Outline

- Iterator
- Containers
- Algorithms

Outline

- Iterator
- Containers
- Algorithms

Iterator

```
1  #include<iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      vector<int> v {32, 68, 57, 39, 86};
7
8      vector<int>::iterator p;
9
10     for (p = v.begin(); p != v.end(); p++) {
11         cout << *p << endl;
12     }
13
14     vector<int>::reverse_iterator rp;
15
16     for (rp = v.rbegin(); rp != v.rend(); rp++) {
17         cout << *rp << endl;
18     }
19 }
```

[lincyu@

32

68

57

39

86

86

39

57

68

32

Practice

- Implement an iterator for Doubly Linked List

Outline

- Iterator
- Containers
- Algorithms

Containers

- Sequence Container
- Stack and Queue
- Set
- Map (Associative Array)

Containers

- Sequence Container
- Stack and Queue
- Set
- Map (Associative Array)

Sequence Container

- `array<>`
- `vector<>`
- `forward_list<>`
- `list<>`
- `deque<>`

forward_list<>

- Singly Linked List
- Advantage
 - Insert a new item almost without cost
- Disadvantage
 - array<> and vector<> are random-access containers
 - forward_list<> and list<> **cannot** support random-access

forward_list<>

```
1  #include<iostream>
2  #include <forward_list>
3  using namespace std;
4
5  int main() {
6      forward_list<int> fl {32, 68, 57, 39, 86};
7
8      forward_list<int>::iterator p;
9
10     for (p = fl.begin(); p != fl.end(); p++) {
11         cout << *p << endl;
12     }
13 }
```

Does not support p-- (Singled Linked List)

list<> (Doubly Linked List) (1/3)

```
1  #include<iostream>
2  #include <list>
3  using namespace std;
4
5  list<int>::iterator findMin(list<int> & fl, list<int>::iterator from) {
6
7      list<int>::iterator checkedItem = from;
8      list<int>::iterator min = from;
9      checkedItem++;
10     do {
11         if (*checkedItem < *min) {
12             min = checkedItem;
13         }
14         checkedItem++;
15     } while (checkedItem != fl.end());
16     return min;
17 }
18 }
```

list<> (Doubly Linked List) (2/3)

```
20 void sortDLL(list<int> & fl) {
21
22     list<int>::iterator wall = fl.begin();
23     list<int>::iterator min;
24
25     do {
26         wall++;
27         if (wall == fl.end()) {
28             break;
29         }
30         wall--;
31         min = findMin(fl, wall);
32         if (min == wall) {
33             wall++;
34             continue;
35         }
36
37         fl.insert(wall, *min);
38         fl.erase(min);
39     } while (wall != fl.end());
40 }
```

list<> (Doubly Linked List) (3/3)

```
42 void printDLLNumber(list<int> & fl) {
43
44     for (list<int>::iterator p = fl.begin(); p != fl.end(); p++) {
45         if (p != fl.begin()) {
46             cout << ' ';
47         }
48         cout << *p;
49     }
50     cout << endl;
51 }
52
53 int main() {
54     list<int> fl {32, 68, 57, 39, 86};
55
56     printDLLNumber(fl);
57     sortDLL(fl);
58     printDLLNumber(fl);
59 }
```

deque (Double Ended Queue)



Source: https://scanftree.com/Data_Structure/duble-ended-queue-dequeue

deque (Double Ended Queue)

```
5  class Node {  
6  public:  
7      Node(int xv, int yv) {  
8          x = xv;  
9          y = yv; }  
10     int x;  
11     int y;  
12 };
```

```
12 int main() {  
13  
14     Node n1(1,1), n2(2,2), n3(3,3);  
15     Node n4(4,4), n5(5,5), n6(6,6);  
16  
17     deque<Node> d {n1, n2, n3, n4};  
18     d.pop_back();  
19     d.push_front(n5);  
20     d.push_back(n6);  
21     d.pop_front();  
22  
23     for (Node node : d) {  
24         cout << "(" << node.x << ", " << node.y << ")" << endl;  
25     }  
26  
27     // Random Access is supported.  
28     cout << "(" << d[2].x << ", " << d[2].y << ")" << endl;  
29 }
```


Practice

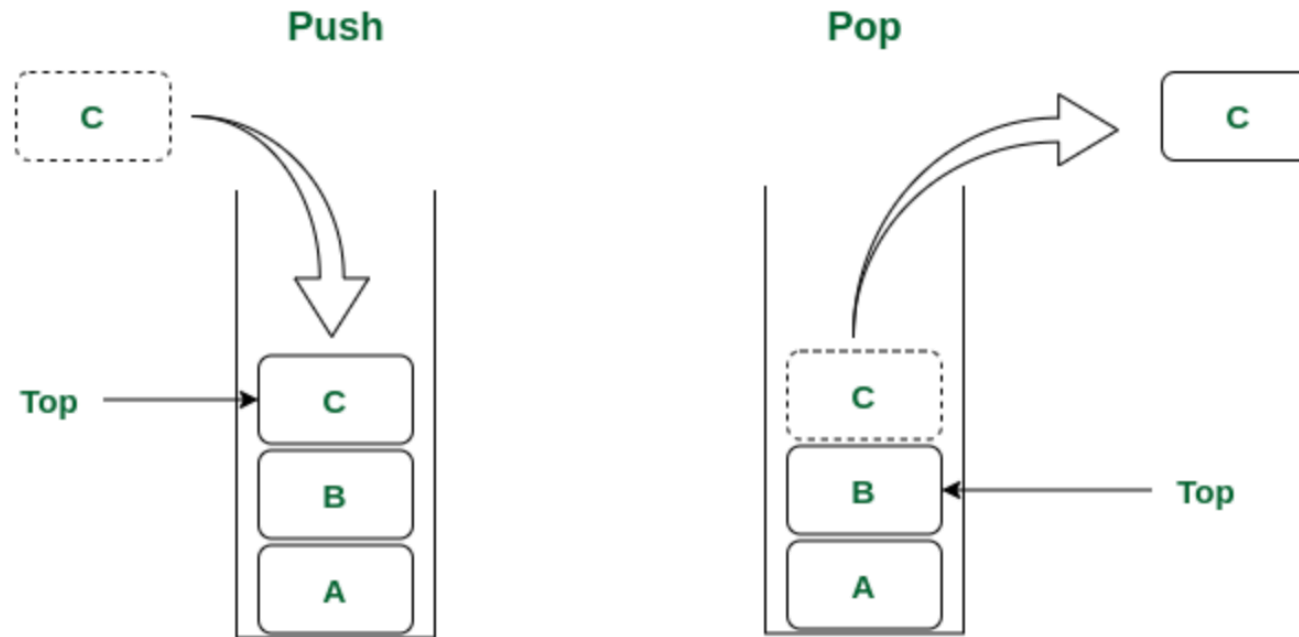
- Design a deque that **can only store 3 elements**. When the deque is full, if `push_back` is executed, one element will be taken out from the front, and if `push_front` is executed, one element will be taken out from the back.

Containers

- Sequence Container
- **Stack and Queue**
- Set
- Map (Associative Array)

Stack

- LIFO (Last In First Out) / FILO (First In Last Out)



Stack Data Structure

Reference: <https://www.geeksforgeeks.org/stack-data-structure/>

Stack

```
1  #include<iostream>
2  #include<stack>
3  using namespace std;
4
5  int main() {
6      stack<int> s1;
7      stack<double> s2;
8
9      s1.push(5);
10     s1.push(6);
11     cout << s1.top() << endl;
12     s1.pop();
13     cout << s1.top() << endl;
14     s1.pop();
15
16     s2.push(3.5);
17     cout << s2.top() << endl;
18     s2.pop();
19 }
```

Queue (Random Access is not supported.)

- FIFO(First In First Out)



Queue Data Structure

Queue

```
5  int main() {
6      queue<int> q;
7      q.push(1);
8      q.push(2);
9      q.push(3);
10
11     cout << q.front() << endl;
12     cout << q.back() << endl;
13
14     /* This for loop has a problem! */
15     for (int i = 0; i < q.size(); i++) {
16         cout << q.front() << ' ';
17         q.pop();
18     } // Only 1 and 2 will be printed.
19     cout << endl;
20 }
```

Practice

- Fix the problem in the previous page so that the program can print all elements in the queue.

Containers

- Sequence Container
- Stack and Queue
- Set
- Map (Associative Array)

set<> (unordered_set<>)

```
5  int main() {
6      std::set<int> myset = {3, 1};
7      myset.insert(2);
8      myset.insert(6);
9      myset.insert(5);
10     myset.insert(3);
11     myset.insert(5);
12
13     for (auto s : myset) {
14         std::cout << s << " ";
15     }
16     std::cout << endl;
17 }
```

```
lincyu@Chih-YudeMacBook-Pro Chapter17 % clang++ -std=c++11 17_05_set.cpp
lincyu@Chih-YudeMacBook-Pro Chapter17 % ./a.out
1 2 3 5 6
lincyu@Chih-YudeMacBook-Pro Chapter17 %
```

Containers

- Sequence Container
- Stack and Queue
- Set
- Map (Associative Array)

map<>

```
5  int main() {
6
7      map<int, string> mapStudent;
8      map<int, string>::iterator iter;
9
10     mapStudent.insert(pair<int, string>(1, "Bob"));
11     mapStudent[2] = "John";
12     mapStudent.insert(pair<int, string>(3, "Bob"));
13     mapStudent.insert(pair<int, string>(1, "Mary"));
14
15     for(iter = mapStudent.begin(); iter != mapStudent.end(); iter++)
16         cout<< iter->first << ' ' << iter->second<< endl;
17 }
18
```

map<>

```
6  int main() {
7      string str {"Hello World!"};
8
9      map<char, int> charCount;
10     map<char, int>::iterator iter;
11
12     for (int i = 0; i < str.length(); i++) {
13         charCount[str[i]] += 1;
14     }
15
16     for (iter = charCount.begin(); iter != charCount.end(); iter++) {
17         cout << iter->first << ": " << iter->second << endl;
18     }
19 }
```

Practice

- Use a map to implement Word Counter

Outline

- Iterator
- Containers
- Algorithms

<algorithm>

- <https://cplusplus.com/reference/algorithm/>

```
1  #include<iostream>
2  #include<algorithm>
3
4  int main() {
5      int arr1[] = {30, 27, 19, 56, 72, 36};
6      double arr2[] = {3.6, 7.2, 8.5, 1.6, 9.1, 5.6};
7
8      std::cout << *std::min_element<int *>(&arr1[0], &arr1[5]) << std::endl;
9      std::cout << *std::min_element<double *>(&arr2[0], &arr2[5]) << std::endl;
10 }
```

std::sort

实习会考！

- <https://shengyu7697.github.io/std-sort/>

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  using namespace std;
5
6  class Point {
7  public:
8      Point (int a, int b)
9          {x = a; y = b;}
10     int x;
11     int y;
12 };
```

```
14 int main() {
15     vector<Point> v = {Point(8,6), Point(5,7), Point(5,3)};
16
17     sort(v.begin(), v.end(),
18         [](Point p1, Point p2) {
19             if (p1.x < p2.x) {
20                 return true;
21             } else if (p1.x == p2.x && p1.y < p2.y) {
22                 return true;
23             }
24             return false;
25         });
26     for (Point p : v) {
27         cout << "(" << p.x << ", " << p.y << ")\n";
28     }
29 }
```